Pranveer Singh Institute of Technology, Kanpur

Odd Semester 2025-26

B.Tech-First Year

Semester-I

Lab File

PROBLEM SOLVING APPROACH

(CS-151)

Submitted To:

Name of Subject Teacher: Puja Kumari

Designation:

Department:

Submitted By:

Name of Student: Mohammad Ahmad

Roll No/ID: 2512080

Branch: CS(core)

INDEX

Lab No.	OBJECTIVES	DATE	MARKS	SIGNATURE
1	Write an algorithm, pseudocode, and draw a flowchart using Raptor **i)** To swap two numbers using a third variable and without a third variable. **ii)** To take input marks of five subjects—physics, chemistry, Biology, mathematics, and computer—calculate percentage and grade according to too given condition- If percentage >= 90%: Grade A If percentage >= 70%: Grade B If percentage >= 70%: Grade B If percentage >= 60%: Grade C If percentage >= 40%: Grade D If percentage <40%: Fail (consider a carryover (CP)) Print the result in the given format: Input 60 76 88 34 61 Output: Pass, 63.80%, Grade C, CP-1			
2	Write an algorithm, pseudocode, and draw a flowchart using Raptor **i)** To check whether the given number is a prime number or not. **ii)** To Check whether the given number is an Armstrong number or not.			

3	Write an algorithm, pseudocode, and draw a flowchart using Raptor **i)** To check whether the given number is a perfect number or not. **ii)** To check for a palindrome.			
4	Write an algorithm, pseudocode, and draw a flowchart using Raptor			

	i) Implement in raptor to Convert decimal into Binary and visa-versa. **ii)** Implement in raptor to Convert decimal into Octal			
5	Write an algorithm, pseudocode, and draw a flowchart using Raptor **i)** Use function which checks an integer as a sum of two prime numbers and prints all possibilities. E.g: Input number: 34 Output: 34 = 3 + 31 34 = 5 + 29 34 = 11 + 23 34 = 17 + 17 **ii)** Find all Armstrong numbers between a range.			
6	Write an algorithm, pseudocode, and recursive C program: i) X^n (power of X) ii) Use recursion to find GCD of two numbers			
7	Write an algorithm, pseudocode, and C program for implementing the following on 1D Array i) Insert ii) Delete			

8	Write an algorithm, pseudocode, and C program for implementing the following searching techniques in 1D-Array i) Linear search ii) Binary search			
9	Write an algorithm, pseudocode, and C program for implementing the following to sort a 1D-Array: i) Bubble ii) Insertion			
10	Write an algorithm, pseudocode, and C program to perform matrix multiplication in a 2D-Array:			

LAB EXPERIMENT 01

I :- TO SWAP TWO NUMBERS USING A THIRD VARIABLE AND WITHOUT A THIRD VARIABLE.

A:- USING THIRD VARIBALE

ALGORITHM:-

1:- Start

2:- Read two number A and B

3:- store A in Temp

4:- Store B in A

5:- Store Temp in B

6:- Display A and B

7:- Stop

PSEUDOCODE:-

START

READ A, B

TEMP ← A

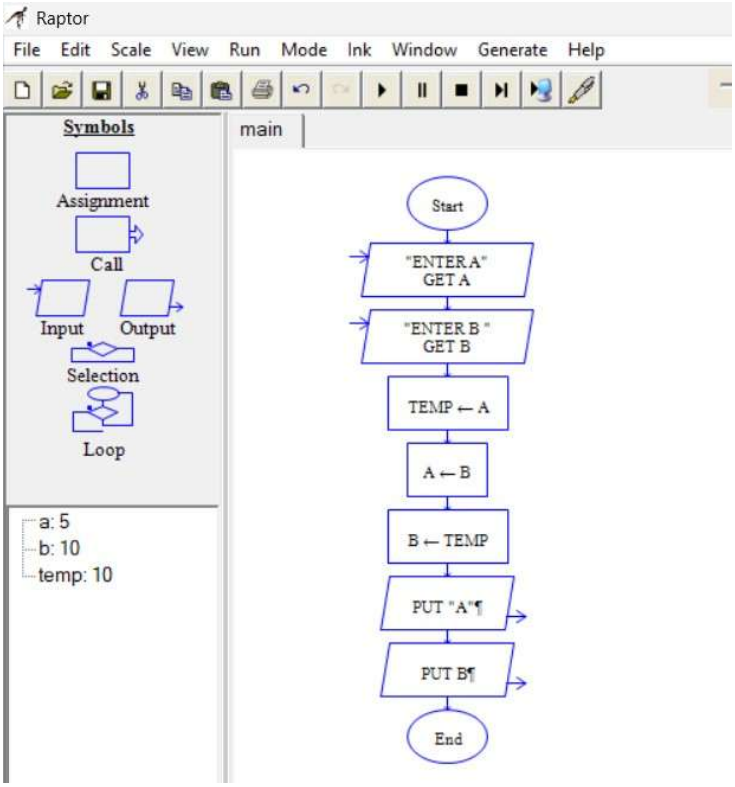
A ← B

B ← TEMP

PRINT A, B

STOP

FLOWCHART:-



B:- WITHOUT USING A THIRD VARIABLE.

ALGORITHM:-

1:- Start

2:- Read A and B

3:- A = A + B **4:-**

B = A − B **5:-** A

= A − B

6:- Display A and B

7:- Stop

PSEUDOCODE:-

START

READ A, B

A ← A + B

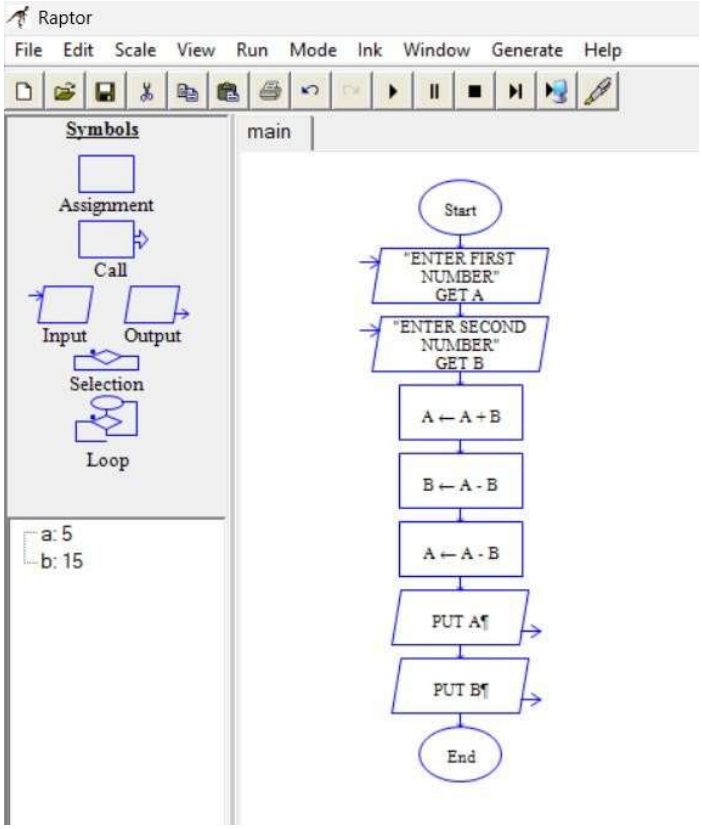
B ← A − B

A ← A − B

PRINT A, B

STOP

FLOWCHART:-



B:- TO TAKE INPUT OF 5 SUBJECTS- "PHYSICS" , "CHEMISTRY", "BIOLOGY", "COMPUTER SCIENCE" AND "MATHEMATICS" – CALCULATE PERCENTAGE AND GRADE ACCORDING TO GIVEN CONDITION

If percentage >= 90%: Grade A If percentage >= 70%: Grade B If percentage >= 70%: Grade B If percentage >= 60%: Grade C If percentage >= 40%: Grade D If percentage

ALGORITHM:-

1:-Start

2:-Read marks of five subjects

3:-Calculate Total and Percentage

4:-Count subjects with marks < 40 (CP count)

5:-Decide Grade based on percentage

6:-If percentage < 40 → Fail

7:-Else → Pass

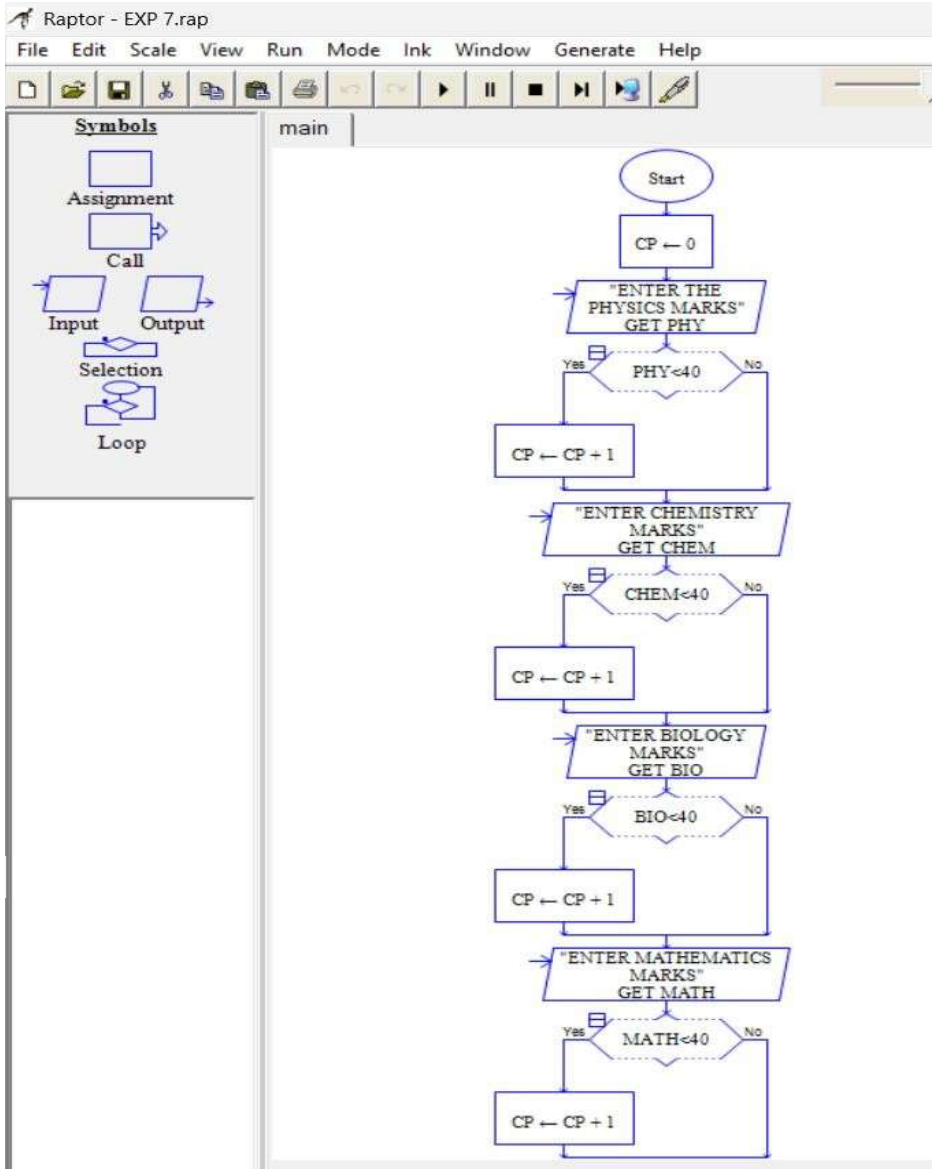
8:-Print result format

9:-Stop

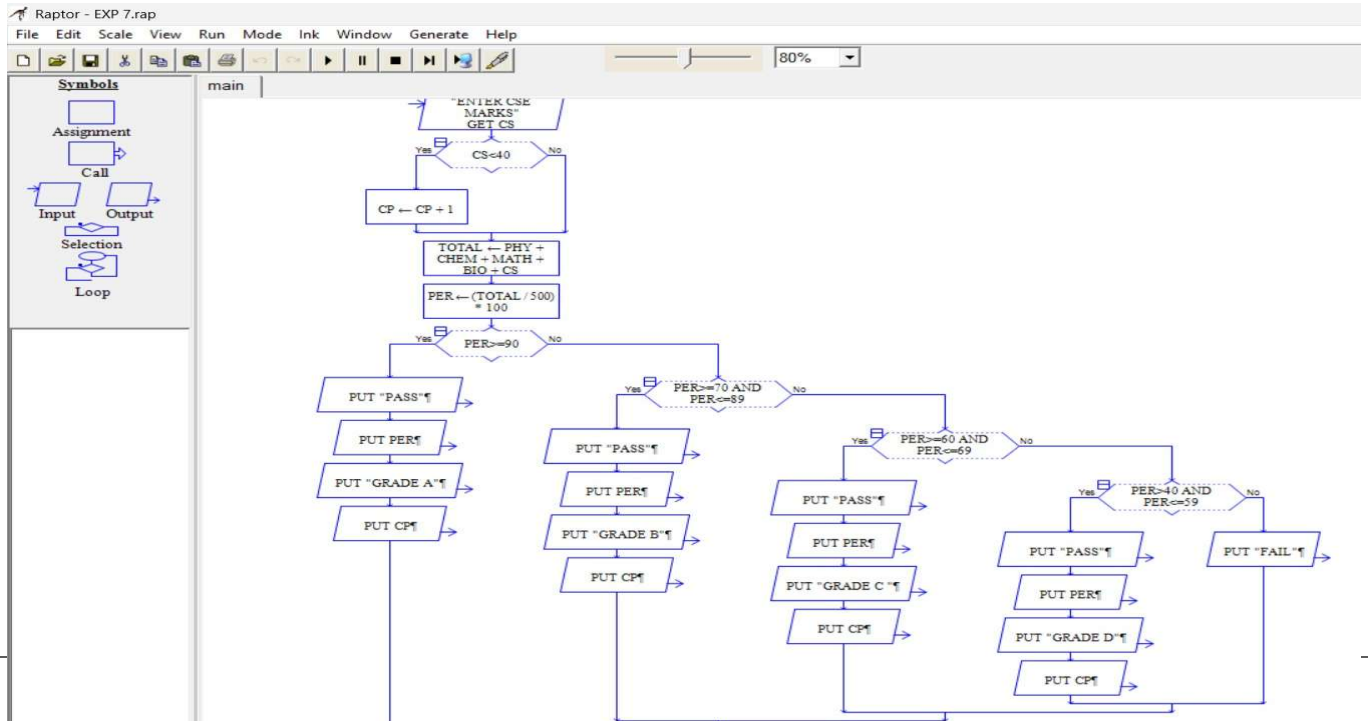
PSEUDOCODE:-

```
START

READ P, C, B, M, CS

TOTAL ← P + C + B + M + CS

PERCENT ← TOTAL / 5

CP ← 0

IF P < 40 THEN CP ← CP + 1

IF C < 40 THEN CP ← CP + 1

IF B < 40 THEN CP ← CP + 1

IF M < 40 THEN CP ← CP + 1

IF CS < 40 THEN CP ← CP + 1

IF PERCENT >= 90
```

```
ELSE
  GRADE ← 'A'

ELSE IF PERCENT >= 70

  GRADE ← 'B'

ELSE IF PERCENT >= 60

  GRADE ← 'C'

ELSE IF PERCENT >= 40

  GRADE ← 'D'


  GRADE ← 'Fail'

IF PERCENT < 40

  RESULT ← 'Fail'


  RESULT ← 'Pass'

PRINT RESULT, PERCENT, GRADE, "CP-", CP
```



FLOWCHART: -



ELSE

STOP

LAB EXPERIMENT 02

A:- TO CHECK WHETHER THE GIVEN NUMBER IS A PRIME NUMBER OR NOT

ALGORITHM:-

1:-START

2:-READ NUMBER n

3:-IF n<=1, THEN ITS NOT PRIME

4:-SET i=2

5:-REPEAT WHILE i<=n/2

- IF n%i==0, THEN NOT PRIME , STOP
- ELSE INCREMENT i

6:-IF NO DIVISOR FOUND, THEN PRIME

7:-STOP

PSEUDOCODE:-

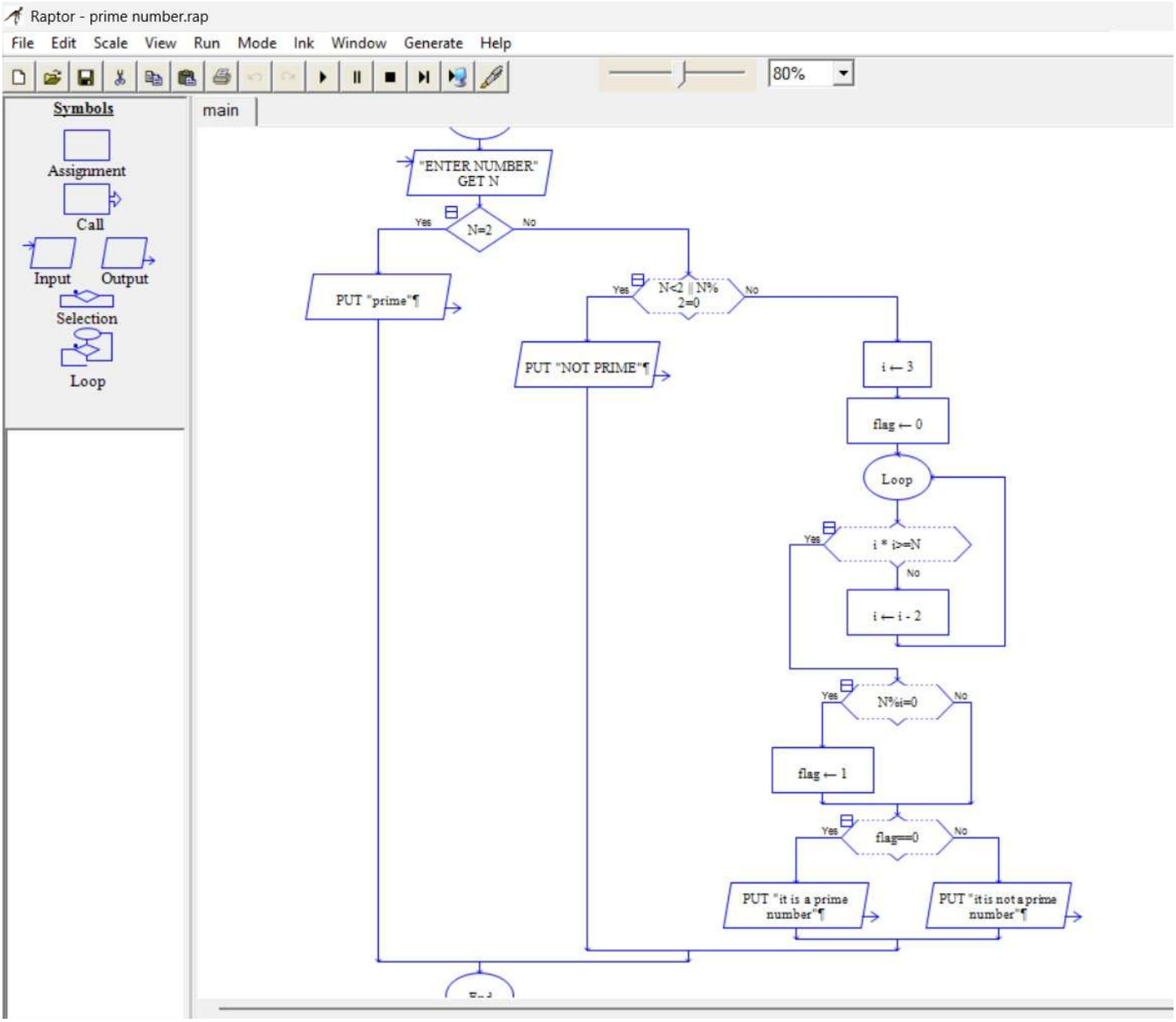
START

READ n

IF n <= 1 THEN

```
   PRINT "Not a Prime Number" ELSE
   i ← 2
flag ← 1
   WHILE i <= n/2 DO
IF n MOD i = 0 THEN
         flag ← 0
         BREAK
ENDIF        i
← i + 1
   ENDWHILE
   IF flag = 1 THEN
      PRINT "Prime Number”
   ELSE
      PRINT "Not a Prime Number"
   ENDIF
ENDIF
STOP
```

FLOWCHART:-



B:- TO CHECK WHETHER THE GIVEN NUMBER IS AN ARMSTRONG OR NOT

ALGORITHM:-

1. START
2. READ NUMBER n
3. STORE n IN temp
4. SET sum=0
5. REPEAT WHILE n>0
    - Digit = n%10
    - Sum=sum+digit^3

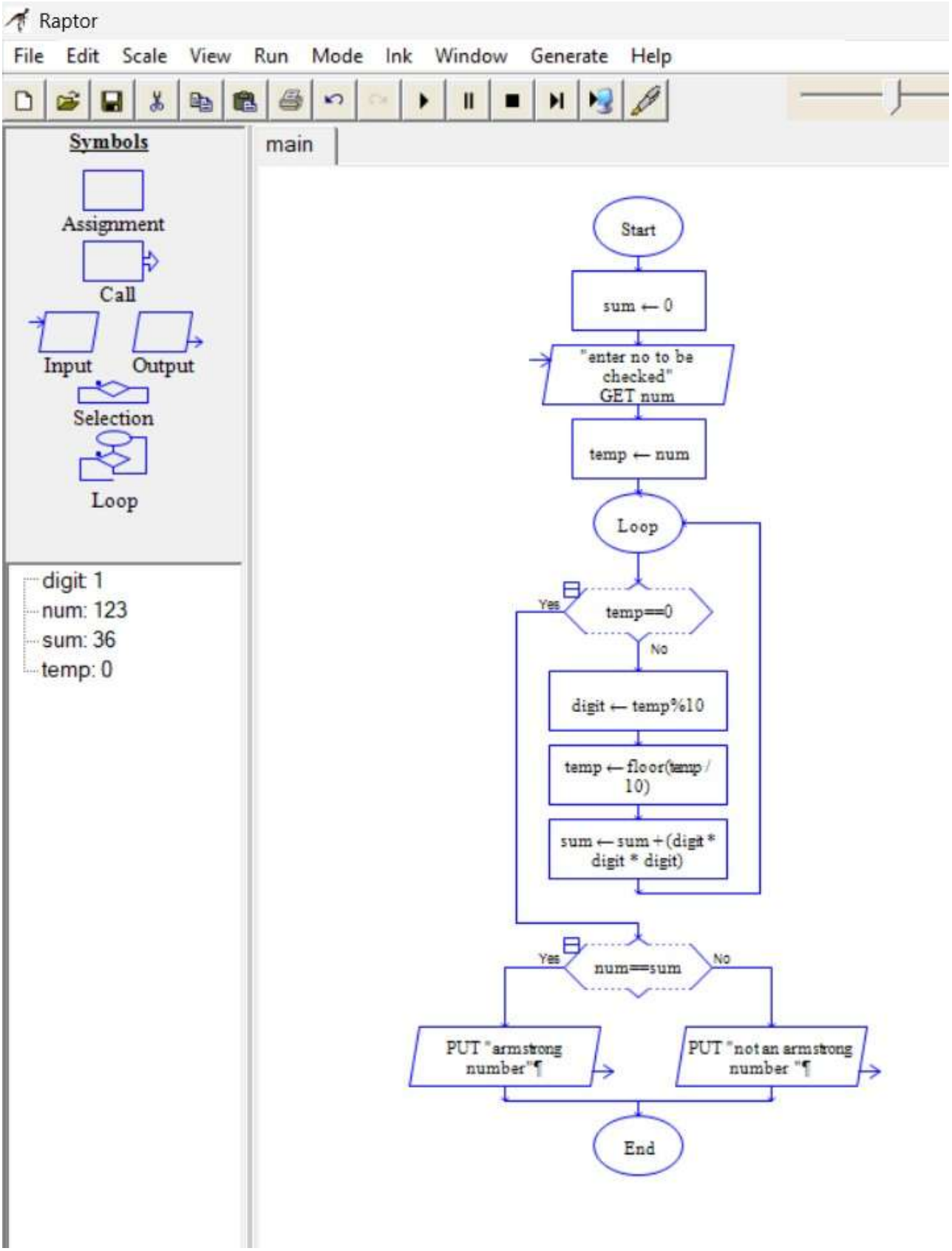
- n=n/10

6. sum==temp, THEN ARMSTRONG
7. ELSE NOT ARMSTRONG
8. STOP

PSEUDOCODE:-

```
START READ n

temp ← n

sum ← 0 WHILE n > 0 DO     digit ←

n MOD 10     sum ← sum + (digit *

digit * digit)     n ← n / 10

ENDWHILE

IF sum = temp THEN

   PRINT "Armstrong Number"

ELSE

   PRINT "Not an Armstrong Number"

ENDIF

STOP
```

FLOWCHART:-


Raptor
File Edit Scale View Run Mode Ink Window Generate Help
Symbols
Assignment
Call
Input
Output
Selection
Loop
main
digit: 1
num: 123
sum: 36
temp: 0
Start
sum ← 0
"enter no to be checked"
GET num
temp ← num
Loop
temp==0
Yes
No
digit ← temp%10
temp ← floor(temp / 10)
sum ← sum + (digit * digit * digit)
num==sum
Yes
No
PUT "armstrong number"¶
PUT "not an armstrong number "¶
End

FLOWCHART:-

LAB EXPERIMENT 03

A:- TO CHECK WHETHER THE GIVEN NUMBER IS A PERFECT NUMBER IS PERFECT NUMBER OR NOT

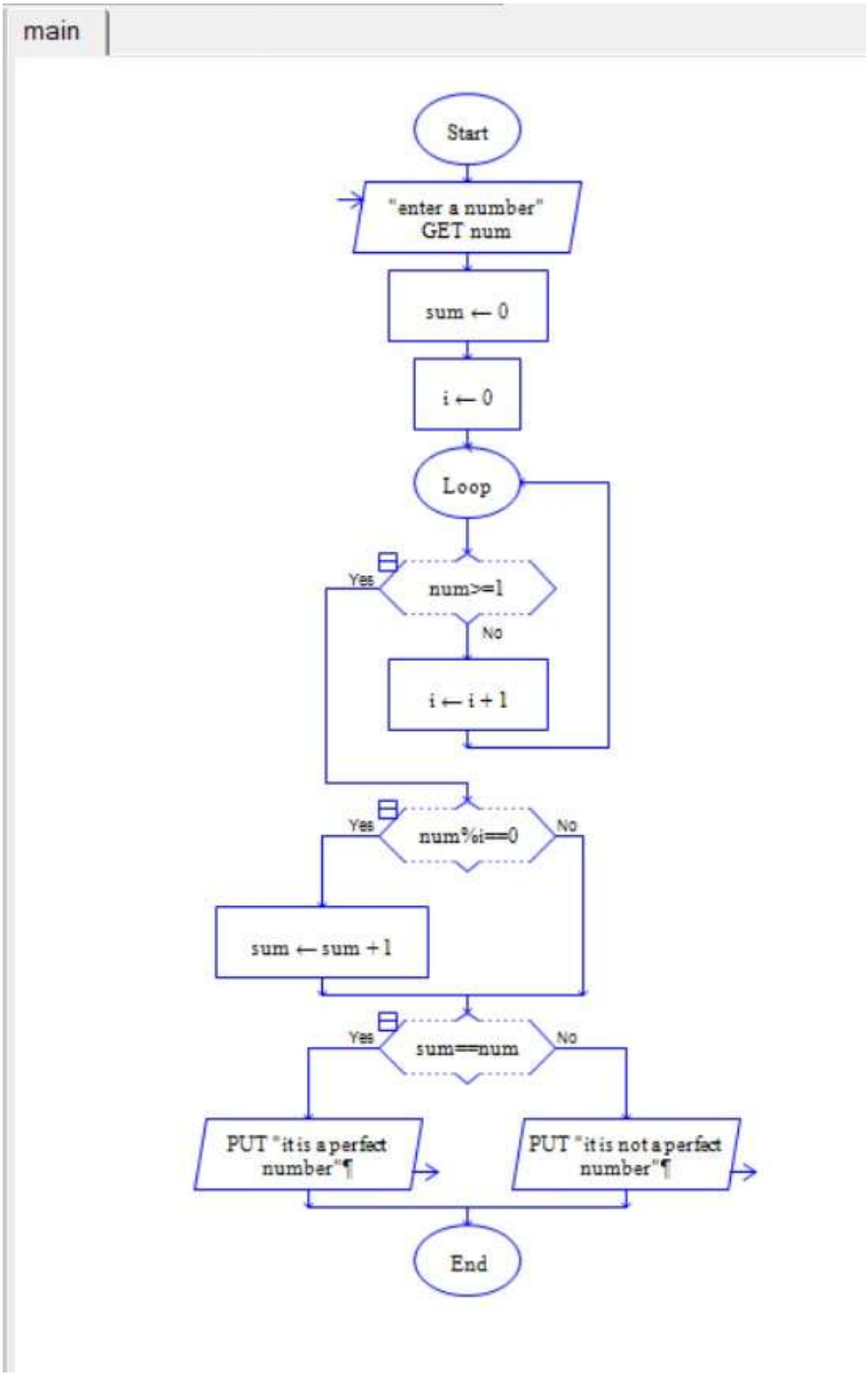
ALGORITHM:-

1. Start
2. Read number n
3. Set sum=0
4. Set i=1
5. Repeat while i<=n/2
    - If n%i==0,then sum=suum+i
    - Increment i
6. If sum==n,print number
7. Else print not a perfect number
8. Stop

PSEUDOCODE:-

```
START READ

n

sum ← 0

i ← 1

WHILE i <= n / 2 DO

IF n MOD i = 0 THEN

sum ← sum + i

ENDIF     i ← i + 1

ENDWHILE

IF sum = n THEN

   PRINT "Perfect Number"

ELSE

   PRINT "Not a Perfect Number"

ENDIF
```

STOP


main
Start
"enter a number"
GET num
sum ← 0
i ← 0
Loop
num>=1
Yes
No
i ← i + 1
num%i==0
Yes
No
sum ← sum + 1
sum==num
Yes
No
PUT "it is a perfect number"¶
PUT "it is not a perfect number"¶
End

FLOWCHART:-

B:- TO CHECK WHETHER A GIVEN NUMBER IS A PALINDROME

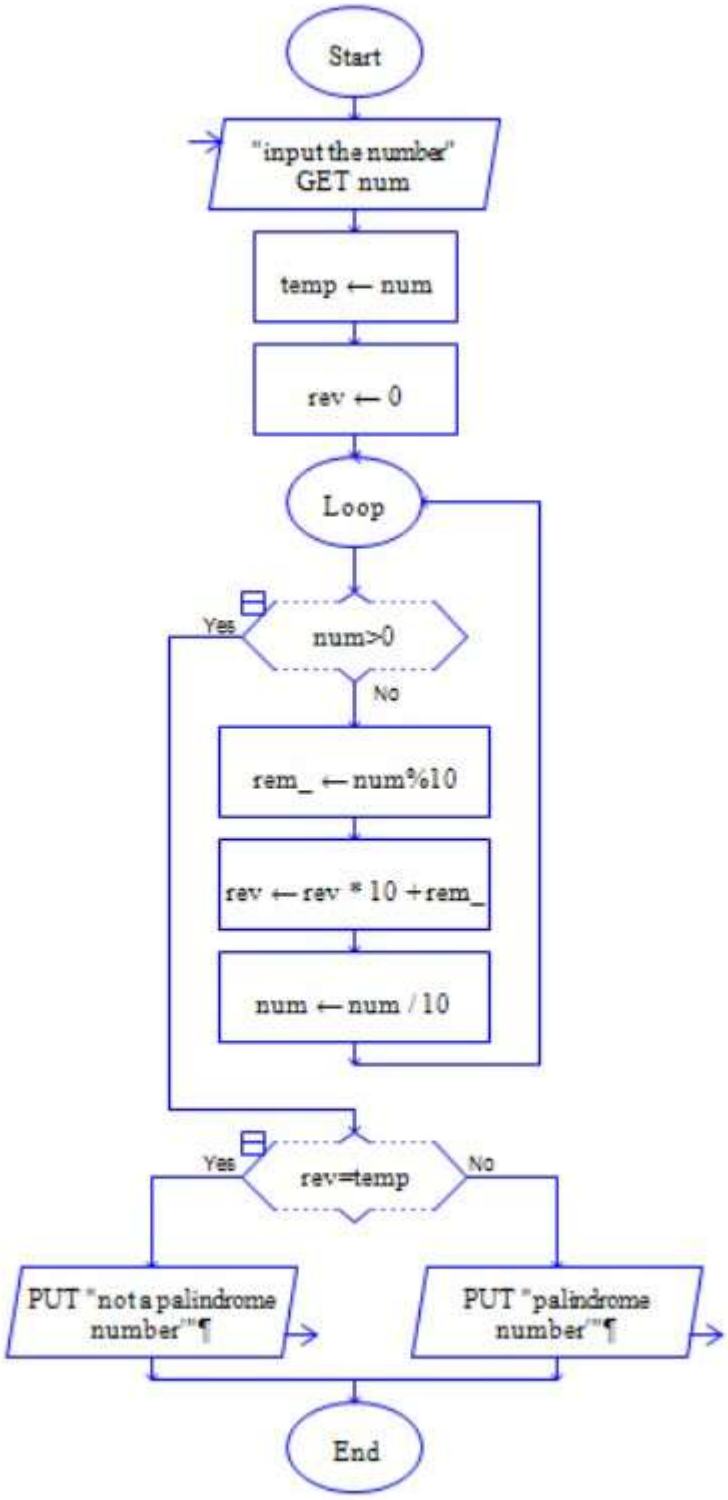
ALGORITHM:-

1. Start
2. Read number n
3. Store n in temp
4. Set rev=0
5. Repeat while n>0
    - Digit=n%10
    - Rev=rev*10+digit
    - n=n/10
6. if rev==temp , print palindrome
7. else print not a palindrome
8. stop

PSEUDOCODE:-

```
START READ n

temp ← n

rev ← 0


WHILE n > 0 DO     digit ← n

MOD 10     rev ← rev * 10 +

digit     n ← n / 10

ENDWHILE


IF rev = temp THEN

   PRINT "Palindrome Number"

ELSE

   PRINT "Not a Palindrome Number"

ENDIF
```

STOP


Start
"input the number"
GET num
temp ← num
rev ← 0
Loop
num>0
Yes
No
rem_ ← num%10
rev ← rev * 10 + rem_
num ← num / 10
rev=temp
Yes
No
PUT "not a palindrome number"¶
PUT "palindrome number"¶
End

FLOWCHART:-

LAB EXPERIMENT 04

A:-IMPLEMENT IN RAPTOR TO CONVERT DECIMAL INTO BINARY AND VISA-VERSA.

ALGORITHM :-

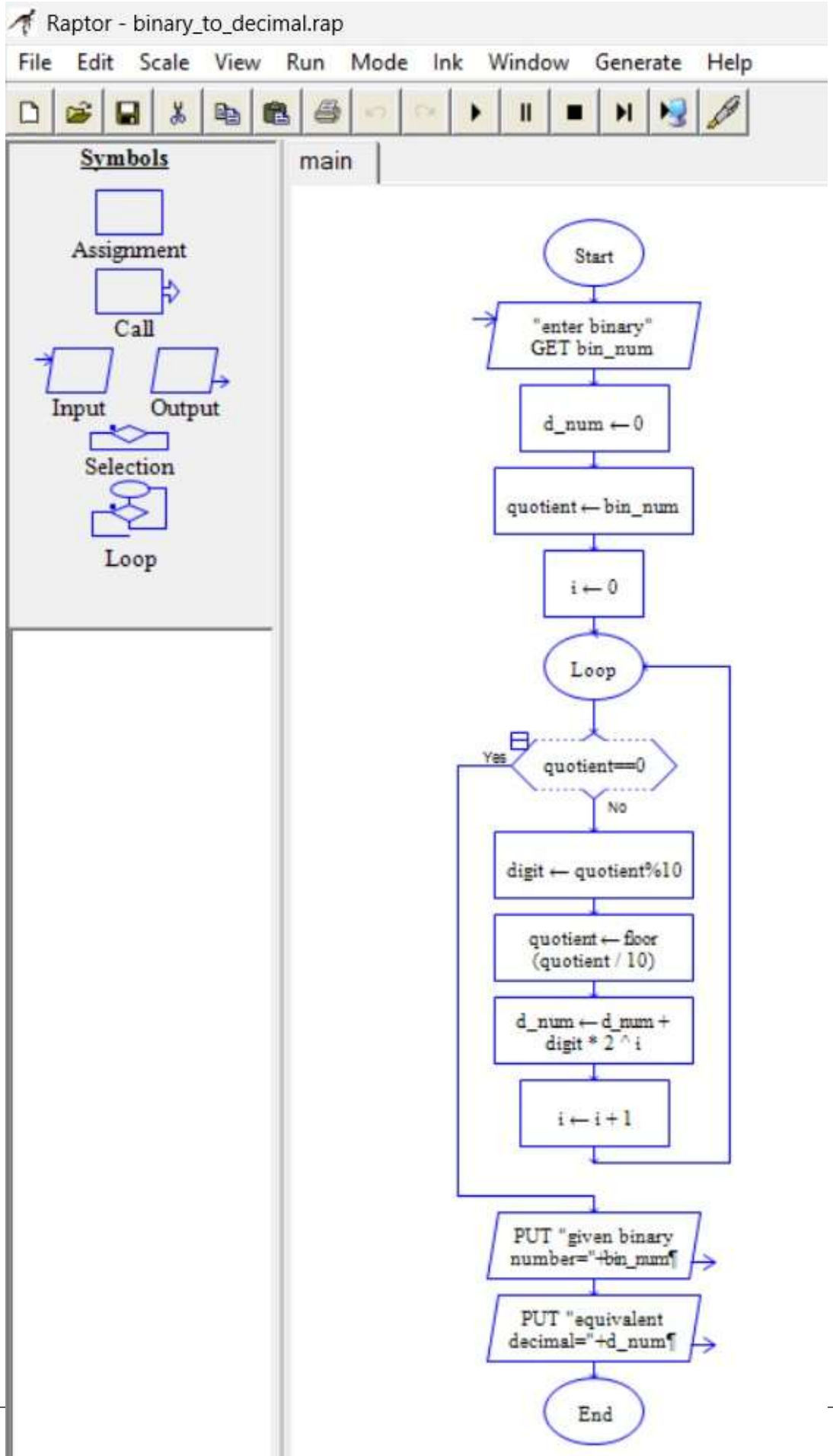
1. Start
2. Read decimal number n
3. Set bin = 0, place = 1
4. While n > 0 remainder = n % 2 bin = bin + remainder × place

   place = place × 10

   n = n / 2
5. Print binary number
6. Stop

PSEUDOCODE:-

```
START READ n

bin ← 0 place ← 1 WHILE

n > 0 DO     rem ← n MOD

2     bin ← bin + rem *

place     place ← place *

10     n ← n / 2

ENDWHILE

PRINT "Binary =", bin

STOP
```


Raptor - binary_to_decimal.rap
File Edit Scale View Run Mode Ink Window Generate Help
Symbols
Assignment
Call
Input
Output
Selection
Loop
main
Start
"enter binary"
GET bin_num
d_num ← 0
quotient ← bin_num
i ← 0
Loop
Yes
quotient==0
No
digit ← quotient%10
quotient ← floor
(quotient / 10)
d_num ← d_num +
digit * 2 ^ i
i ← i + 1
PUT "given binary
number="+bin_num¶
PUT "equivalent
decimal="+d_num¶
End

FLOWCHART:-

B:- IMPLEMENT IN RAPTOR TO CONVERT DECIMAL INTO OCTAL .

ALGORITHM:-

1. Start
2. Read decimal number n
3. Set oct = 0, place = 1
4. While n > 0
   - remainder = n % 8
   - oct = oct + remainder × place
   - place = place × 10
   - n = n / 8
5. Print octal number
6. Stop

PSUEDOCODE:-

START

READ n oct ← 0 place ←

1 WHILE n > 0 DO rem

← n MOD 8 oct ← oct

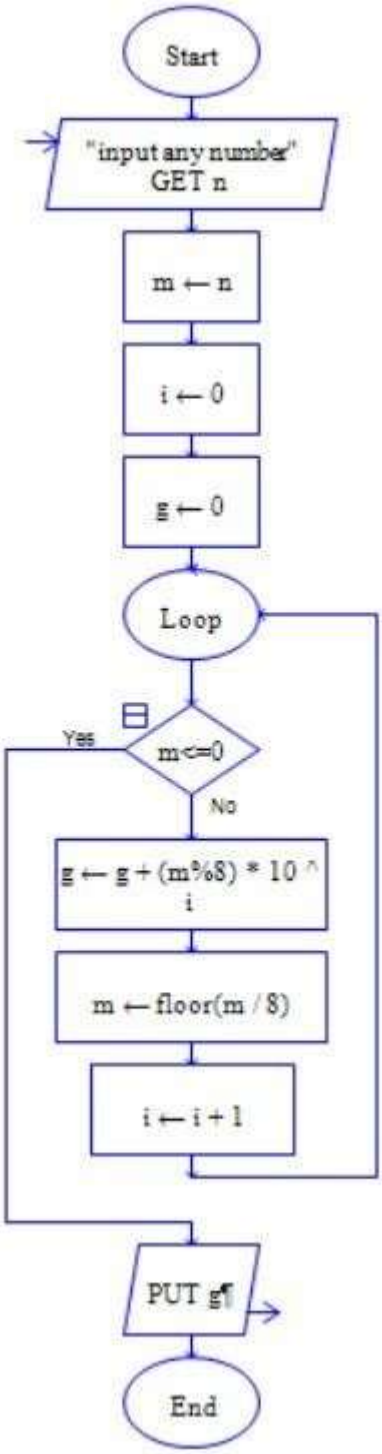
+ rem * place place ←

place * 10 n ← n / 8

ENDWHILE

PRINT "Octal =", oct

STOP


Start
"input any number"
GET n
m ← n
i ← 0
g ← 0
Loop
Yes
m<=0
No
g ← g + (m%8) * 10 ^ i
m ← floor(m / 8)
i ← i + 1
PUT g¶
End

FLOWCHART:-

Lab experiment 05

A:-Use function which checks an integer as a sum of two prime numbers and prints all possibilities

Algorithm:-

1. Start
2. Read integer n
3. For i from 2 to n/2

- Check if i is prime
- Check if n - i is prime
- If both are prime, print n = i + (n - i)

4. Stop

PSEUDOCODE:-

```
START
READ n
FOR i ← 2 TO n/2 DO
  IF isPrime(i) AND isPrime(n - i) THEN
    PRINT n, "=", i, "+", (n - i)
  ENDIF
ENDFOR
STOP
FUNCTION isPrime(x)
  IF x <= 1 THEN
    RETURN false
  ENDIF
  FOR j ← 2 TO x/2 DO
    IF x MOD j = 0 THEN
      RETURN false
```

```
    ENDIF
ENDFOR
RETURN true
```

END FUNCTION

C PROGRAM CODE:-

```
#include <stdio.h>

int isPrime(int x); int

main()

{    int n,

i;

   printf("Enter a number: ");

scanf("%d", &n);     for(i = 2;

i <= n/2; i++)

   {

      if(isPrime(i) && isPrime(n - i))

      {

         printf("%d = %d + %d\n", n, i, n - i);

      }

   }

   return 0;

}

int isPrime(int x)

{    int j;     if(x <= 1)

return 0;     for(j = 2; j <=

x / 2; j++)

   {        if(x % j

== 0)

return 0;

   }

   return 1;

}
```

B:- FIND ALL ARMSTRONG NUMBERS BETWEEN A RANGE

ALGORITHM:-

1. start
2. read start and end of range
3. for each number I from start to end
   - count number of digit in i
   - find sum of each digit raised to the power of digit count
   - if sum equal i , print i
4. stop

PSEUDOCODE:-

```
START
INPUT start, end FOR num ←
start TO end    temp ← num
count ← 0    WHILE temp ≠ 0
temp ← temp / 10       count ←
count + 1    END WHILE    temp ←
num    sum ← 0    WHILE temp ≠
0       digit ← temp % 10       sum
← sum + digit^count       temp ←
temp / 10
  END WHILE
  IF sum = num THEN
    PRINT num
  END IF
END FOR
STOP
```

C PROGRAM CODE:-

```
#include <stdio.h>

#include <math.h> int

main()

{

   int start, end, num, temp, digit, sum, count;     printf("Enter starting range: ");

scanf("%d", &start);     printf("Enter ending range: ");     scanf("%d", &end);

printf("Armstrong numbers between %d and %d are:\n", start, end);     for(num =

start; num <= end; num++)

   {

      temp = num;

count = 0;

while(temp != 0)

      {

         temp = temp / 10;           count++;

      }

      temp = num;

sum = 0;

while(temp != 0)

      {

         digit = temp % 10;           sum = sum +

pow(digit, count);           temp = temp / 10;

      }

      if(sum == num)

      {

         printf("%d ", num);

      }

   }

   return 0;

}
```

LAB EXPERIMENT 06

A:- WRITE A ALGORITHM,PSEUDOCODE AND RECRSIVE C CODE FOR X^n (POWER OF X)

ALGORITHM:-

1. start
2. read value of x and n
3. if n==0, return 1
4. otherwise return x* power(x,n-1)
5. display result
6. stop

PSEUDOCODE:-

```
START

INPUT X, n

FUNCTION power(X, n)

   IF n == 0 THEN

      RETURN 1

   ELSE

      RETURN X * power(X, n - 1)

   END IF

END FUNCTION

result ← power(X, n)

PRINT result

STOP
```

C PROGRAM CODE:-

```
#include <stdio.h>


int power(int x, int n)

{
```

```
   if(n == 0)        return 1;
else        return x * power(x, n
- 1);
}

int main()
{    int x,
n;

   printf("Enter value of X: ");
scanf("%d", &x);

   printf("Enter value of n: ");
scanf("%d", &n);

   printf("Result = %d", power(x, n));

   return 0;
}
```

B:- FIND GCD OF TWO NUMBERS USING RECURSION

ALGORITHM:-

1. start
2. read two numbers a and b
3. if b==0, return a
4. else return gcd(b,a%b)
5. display gcd

6. stop

PSEUDOCODE:-

```
Start
Input a, b
Function gcd(a, b)
IF b == 0 then
      return a
   Else
      Return gcd(b, a MOD b)
   End if
End function
result ← gcd(a, b)
print result
Stop
```

Recursive c program code:-

```
#include <stdio.h>
int gcd(int a, int b)
{
   if(b == 0)        return
a;     else        return
gcd(b, a % b);
```

```
}
int main()
{    int a,
b;
   printf("Enter two numbers: ");
scanf("%d %d", &a, &b);
printf("GCD = %d", gcd(a, b));
return 0;
}
```

LAB EXPERIMENT 07

A:- WRITE AN ALGORITHM,PSEUDOCODE, AND C PROGRAM FOR IMPLEMENTING THE FOLLOWING ON 1D ARRAY

:::INSERT ALGORITHM:-

1. start
2. read number of element n
3. read array elements
4. read element item and position pos
5. shift elements from right to left starting from index n to pos
6. insert item at position pos
7. increase array size
8. display updated array
9. stop

PSEUDOCODE:-

```
Start
Input n
For i ← 0 TO n-1
```

Input arr[i]

End for

Input item, pos For i

← n downto pos

arr[i] ← arr[i-1]

End for arr[pos-1]

← item n ← n + 1

Print updated array

Stop

C PROGRAM CODE:-

```
#include <stdio.h> int
main()
{
   int arr[100], n, i, item, pos;
printf("Enter number of elements: ");
scanf("%d", &n);     printf("Enter array
elements:\n");
   for(i = 0; i < n; i++)
scanf("%d", &arr[i]);     printf("Enter
element to insert: ");     scanf("%d",
&item);     printf("Enter position: ");
scanf("%d", &pos);
   for(i = n; i >= pos; i--)
arr[i] = arr[i - 1];
```

```
   arr[pos - 1] = item;     n++;
printf("Array after insertion:\n");
   for(i = 0; i < n; i++)
printf("%d ", arr[i]);     return
0;
}
```

II. DELETION ALGORITHM:-

1. start
2. read number of element n
3. read array element
4. read position pos to delete
5. shift element from left to right starting from pos
6. decrease array size
7. display updated array
8. stop

PSEUDOCODE:-

```
Start
Input n
For i ← 0 TO n-1
   Input arr[i]
End for
Input pos
For i ← pos-1 TO n-2
   arr[i] ←
arr[i+1] End for  n
← n - 1
```

Print updated array

Stop

C PROGRAM CODE:-

```
#include <stdio.h>
int main()
{
   int arr[100], n, i, pos;
   printf("Enter number of elements:
");    scanf("%d", &n);    printf("Enter
array elements:\n");
   for(i = 0; i < n; i++)
scanf("%d", &arr[i]);    printf("Enter
position to delete: ");    scanf("%d",
&pos);
   for(i = pos - 1; i < n - 1;
i++)       arr[i] = arr[i + 1];
n--;
   printf("Array after deletion:\n");
   for(i = 0; i < n; i++)
printf("%d ", arr[i]);    return
0;
```

}

LAB EXPERIMENT 08

WRITE AN ALGORITHM, PSEDOCODE AND C PROGRAM FOR IMPLEMENTING THE FOLLOWING ON 1D ARRAY

A:- LINEAR SEARCH

ALGORITHM:-

1. Start
2. Read number of elements n
3. Read array elements
4. Read element key to search
5. Compare key with each element of array
6. If match found, display position and stop
7. If not found after loop, display “Not Found”
8. Stop

PSEUDOCODE:-

```
START
INPUT n
FOR i ← 0 TO n-1
   INPUT arr[i]
END FOR
INPUT key flag
← 0
FOR i ← 0 TO n-1
   IF arr[i] == key THEN
      PRINT "Element found at position", i+1
flag ← 1        BREAK
```

```
   END IF
END FOR
IF flag == 0 THEN
   PRINT "Element not found"
END IF
STOP
```

C PROGRAM CODE:-

```
#include <stdio.h> int
main()
{    int arr[100], n, i, key, flag = 0;
printf("Enter number of elements: ");
scanf("%d", &n);    printf("Enter array
elements:\n");    for(i = 0; i < n; i++)
scanf("%d", &arr[i]);    printf("Enter
element to search: ");    scanf("%d",
&key);    for(i = 0; i < n; i++)
   {
      if(arr[i] == key)
      {
         printf("Element found at position %d", i + 1);
         flag = 1;
break;
      }
   }
   if(flag == 0)
printf("Element not found");
return 0;}
```

B:- BINARY SEARCH

ALGORITHM:-

1. Start
2. Read number of elements n
3. Read sorted array elements
4. Read element key
5. Set low = 0, high = n-1
6. While low ≤ high:
7. mid = (low + high) / 2
8. If arr[mid] == key, display position and stop
9. If key < arr[mid], set high = mid - 1
10. Else set low = mid + 1
11. If not found, display “Not Found”
12. Stop

PSEUDOCODE:-

```
START

INPUT n

FOR i ← 0 TO n-1

   INPUT arr[i]

END FOR INPUT key low

← 0 high ← n - 1 flag ←

0 WHILE low ≤ high

mid ← (low + high) / 2

   IF arr[mid] == key THEN

      PRINT "Element found at position", mid+1

      flag ← 1

      BREAK
```

```
   ELSE IF key < arr[mid] THEN
high ← mid - 1     ELSE
low ← mid + 1     END IF
END WHILE
IF flag == 0 THEN
   PRINT "Element not found"
END IF
STOP
```

C PROGRAM CODE :-

```
#include <stdio.h> int
main()
{    int arr[100], n, i, key;    int low, high,
mid, flag = 0;    printf("Enter number of
elements: ");    scanf("%d", &n);
printf("Enter sorted array elements:\n");
for(i = 0; i < n; i++)        scanf("%d",
&arr[i]);    printf("Enter element to
search: ");    scanf("%d", &key);    low = 0;
high = n - 1;    while(low <= high)
   {
      mid = (low + high) / 2;
if(arr[mid] == key)

      {
         printf("Element found at position %d", mid + 1);
         flag = 1;
break;
      }
```

```
      else if(key < arr[mid])
high = mid - 1;        else
low = mid + 1;
   }
   if(flag == 0)
printf("Element not found");
return 0;
}
```

LAB EXPERIMENT 09

WRITE AN ALGORITHM , PSEUDOCODE , AND C PROGRAM FOR IMPLEMENTING THE FOLLOWING SEACHING TECHNIQUES IN 1D ARRAY

A:- BUBBLE SORTING

ALGORITHM:-

1. Start
2. Read number of elements n
3. Read array elements
4. Repeat steps for i = 0 to n-2
5. Compare adjacent elements
6. If arr[j] > arr[j+1], swap them
7. After each pass, largest element moves to the end
8. Display sorted array
9. Stop

PSEUDOCODE:-

START

```
INPUT n

FOR i ← 0 TO n-1

   INPUT arr[i]

END FOR

FOR i ← 0 TO n-2

   FOR j ← 0 TO n-2-i        IF

arr[j] > arr[j+1] THEN

temp ← arr[j]            arr[j]

← arr[j+1]            arr[j+1]

← temp

       END IF

   END FOR

END FOR

PRINT sorted array

STOP
```

C PROGRAM CODE:-

```
#include <stdio.h>

int main()

{    int arr[100], n, i, j, temp;

printf("Enter number of elements: ");

scanf("%d", &n);     printf("Enter array

elements:\n");     for(i = 0; i < n; i++)

scanf("%d", &arr[i]);     for(i = 0; i < n -

1; i++)

   {        for(j = 0; j < n - 1 - i;

j++)

       {            if(arr[j] >

arr[j + 1])

           {
```

```
            temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp;
          }
      }
   }
   printf("Sorted array (Bubble
Sort):\n");     for(i = 0; i < n; i++)
printf("%d ", arr[i]);     return 0;
}
```

B:- INSERTION SORT

ALGORITHM:-

1. Start
2. Read number of elements n
3. Read array elements
4. Assume first element is already sorted
5. Pick next element and insert it into correct position in sorted part
6. Repeat until array is sorted
7. Display sorted array
8. Stop

PSEUDOCODE:-

```
START

INPUT n

FOR i ← 0 TO n-1

   INPUT arr[i]

END FOR

FOR i ← 1 TO n-1

   key ← arr[i]

   j ← i - 1

   WHILE j ≥ 0 AND arr[j] > key

      arr[j+1] ← arr[j]

      j ← j - 1

   END WHILE

   arr[j+1] ← key

END FOR

PRINT sorted array
```

STOP

C PROGRAM CODE:-

```
#include <stdio.h> int
main()
{
   int arr[100], n, i, j, key;
   printf("Enter number of elements: ");
scanf("%d", &n);     printf("Enter array
elements:\n");
   for(i = 0; i < n; i++)
scanf("%d", &arr[i]);     for(i =
1; i < n; i++)
   {
      key = arr[i];
      j = i - 1;
      while(j >= 0 && arr[j] > key)
      {
         arr[j + 1] = arr[j];
         j--;
      }
      arr[j + 1] = key;
   }
   printf("Sorted array (Insertion Sort):\n");
```

```
   for(i = 0; i < n; i++)
printf("%d ", arr[i]);     return
0;

}
```

LAB EXPERIMENT 10

WRITE AN ALGORITHM , PSEUDOCODE , AND C PROGRAM TO PERFORM MATRIX MULTIPLICATION IN A 2D ARRAY

ALGORITHM:-

1. Start
2. Read rows and columns of Matrix A (r1, c1)
3. Read rows and columns of Matrix B (r2, c2)
4. If c1 ≠ r2, print “Multiplication not possible” and stop
5. Read elements of Matrix A and Matrix B
6. Multiply matrices using three nested loops
7. Store result in Matrix C
8. Display Matrix C
9. Stop

PSEUDOCODE:-

```
START
INPUT r1, c1
INPUT r2, c2
IF c1 ≠ r2 THEN
   PRINT "Matrix multiplication not possible"
   STOP
END IF
```

```
FOR i ← 0 TO r1-1
   FOR j ← 0 TO c1-1
      INPUT A[i][j]
   END FOR
END FOR
FOR i ← 0 TO r2-1
   FOR j ← 0 TO c2-1
      INPUT B[i][j]
   END FOR
END FOR
FOR i ← 0 TO r1-1
   FOR j ← 0 TO c2-1
      C[i][j] ← 0
      FOR k ← 0 TO c1-1
         C[i][j] ← C[i][j] + A[i][k] * B[k][j]
      END FOR
   END FOR
END FOR
PRINT Matrix C
STOP
```

C PROGRAM CODE :-

```
#include <stdio.h> int
main()
{
   int A[10][10], B[10][10], C[10][10];
int r1, c1, r2, c2;     int i, j, k;
   printf("Enter rows and columns of Matrix A: ");
scanf("%d %d", &r1, &c1);     printf("Enter rows
and columns of Matrix B: ");     scanf("%d %d",
&r2, &c2);
```

```
   if(c1 != r2)
   {
      printf("Matrix multiplication not possible");
return 0;

   }
   printf("Enter elements of Matrix A:\n");
for(i = 0; i < r1; i++)        for(j = 0; j < c1;
j++)            scanf("%d", &A[i][j]);
printf("Enter elements of Matrix B:\n");
for(i = 0; i < r2; i++)        for(j = 0; j < c2;
j++)            scanf("%d", &B[i][j]);     for(i =
0; i < r1; i++)
   {
      for(j = 0; j < c2; j++)
      {
         C[i][j] = 0;
for(k = 0; k < c1; k++)
         {
            C[i][j] += A[i][k] * B[k][j];
         }
      }
   }
   printf("Resultant Matrix:\n");
for(i = 0; i < r1; i++)
   {
      for(j = 0; j < c2; j++)
printf("%d ", C[i][j]);
printf("\n");
   }
   return 0;
```

}